

Managing Changes in Specification-Driven Development

An investigation into methods to update working, specification-delivered software.

Ed Barlow Web Cloud Studio July 2026



Abstract

In specification-driven development (SDD) the specification is the source of truth and the code is generated from it.

But there is no best practice method to update your working software while keeping specifications in sync with changes. There is no best practice way to manage emergency bug fixes. There is no best practice way to manage normal changes.

This paper documents my investigation into testing various methodologies to determine what works best.

Keywords: specification-driven development, change management, dependency tracking, typed encapsulation, content-hash staleness, iteration

Concept #1 — Use LLM Transaction Log

I am a database guy, so my first instinct was to simply create a transaction log of changes that I was manually applying to my code base. I created a tool to read Claude session transcripts and logs, and to synthesize the changes directly into change tickets with a timestamp for ordering.

Concept #1 failed on signal-to-noise. A working session contains exploration, dead ends, and thinking. Extracting "the changes" from that stream produced slop: tickets that could not reproduce the software accurately, and it diverged at a steady rate. Bugs were not fixed in the parallel environment, and changes that displayed correctly in the primary were not consistently reflected in the rebuild. The build process simply could not be trusted. I am sure we could do better, but this method seemed error-prone in its best case, and my investigation led me to believe that this approach was a dead end.

Lesson: Mining change tickets out of the LLM is hard, and the specs will diverge from the code.

Concept #2 — LLM Created Change Logs

The second concept had the LLM express changes into a JSON change log. This recorded impactful changes to the codebase. It basically excluded the noise from Concept #1 and recorded summaries of the prompts that impacted the code, recording date, type, scope, and description of the changes. A program later read the log, grouped entries by scope, and applied each change in order.

This was more honest than mining logs, but clunky in practice. The change log was a second place to look, separate from the specification, and changes were queued together whenever the apply-change software was run. LLMs are not deterministic, and the change logs did not completely and accurately reflect the actual changes made, plus there was no human-readable artifact of tasks to do. And finally, the specification therefore did not reflect the application — it was not the only source of truth — the specs plus the change log was. Drift AND harder-to-read specs.

Lesson: *A change log is a queue, and a queue is latency. The longer a change sits described but not applied, the more the specification lies about the system.*

Concept #3 — Edit the Specification and Rebuild

Specs are the source of truth, so the next try was to edit the specification directly and rebuild. There was complexity revealed in implementation.

Experience 1: A database column change from null to not null forced a complete rebuild. Foundational files are required context for every specification. A trivial change that should have had zero impact became a full rebuild.

Lesson: *Constitution changes invalidate everything so - by definition - changing them can not invalidate build steps*

Lesson: *Changes to Foundational Specifications have a HUGE impact. If you change your constitution, your foundation, or your persistence layer, you can force a full rebuild.*

Experience 2: Some specification changes may cause rebuilds to other specifications. You need to know. For example, a change to a web route could force a rebuild of dependent screens. You need to know the impact of your changes to rebuild correctly. Additionally, sometimes those changes do not invalidate anything - for example in the above database column change - how do you know if its significant enough to force a rebuild and if it is not.

Lesson: There seems no point in re-providing the original full specification when a change ticket provides the same details. That's bloat and you need to send a before and after image from the git logs. It's easier and just as correct to send only a change ticket.

Lesson: You must KNOW how to rebuild — you can't just rebuild the changed specifications. You need to understand which objects are impacted by your changed specifications and build them too.

Concept #4 — Change Tickets

So now we have change tickets — well-defined edits that change software behavior, fed to your build process. If you know how to rebuild from the impact (see later), it is minimal context and works.

Lesson: Change Tickets need ordering to preserve the build — just sequentially number them (or timestamp them).

Concept #5 — Controllable Updates

Lastly, I implemented a mechanism (using skills) that let me make a plan from which I could a) update to code, b) update the specification, or c) do both.

- **Create the change ticket first, then rebuild later.**
- **Update the software first.** If you update code and write a change ticket, you theoretically have a durable change ticket already applied, but your specifications WILL drift.
- **Update the specification and software together.** Edit the specification, dirty the downstream, then apply the change to the code in the same session. This is the same process as creating the change ticket first and then rebuilding immediately.

Lesson:: This proved to me that Concept #4 is the correct solution. Use Change Tickets. My rationale was that two of the above three paths are identical, and one does not work. So... why bother with the complexity? Just write your change down in a change ticket file (with a sequence number) and then feed it to the LLM. Occam's Razor.

Solution #1 — Encapsulation

Some kinds of specification changes invalidate your builds. The obvious solution is to encapsulate those changes or to require foundational changes as incremental changes (but that still has impact).

Many database deployment tools, such as Liquibase, simply order their change queue. That forces the build to be reproducible.

Encapsulate using libraries - for example with a persistence library - means that you can change how a feature works without impacting downstream objects. The only exception is if the contract changes. Downstream objects are impacted only if the behavior changes.

Lesson: *Detecting file changes is not enough to determine the impact on dependent objects - you need to understand if the contract changes.*

This led to the concept of compaction. A compact version of a specification is its contract. You should be able to detect contract changes - but for that you need a deterministic way to compact that file into its contract. You only need to change dependent files if the contract changes.

Encapsulation almost guarantees that the compacted contract for the new persistence library will not change due to minor changes.

For example: Encapsulation/Compaction of your infrastructure layer will reduce the impact on service specifications. Encapsulation/Compaction of the service layer will reduce the impact on the User Interface. To generalize that discipline to every persistent store:

- Each database table is a typed row plus a CRUD class, composed into one `Database` class.
- All environment access goes through one typed `Config` class.
- File access goes through a `FileStore` class.
- External services are wrapped in classes over a shared base class.

Encapsulation enables clear boundaries. Dependent objects can use the contract, which is a compressed version of the foundational specification. Only if that contract changes does the dependent object need to rebuild.

LLM-generated compaction is NOT generally deterministic. A single byte change to the contract will indicate that downstream objects need to rebuild. That rebuild may be unneeded. VERY strict definitions of the contract compaction are required to avoid this — do not put LLM-created strings into it. There is no need, for example, for a description field in the contract — just how things are called.

Lesson: *The contract for the specification, not the specification, is the dependency boundary. Detecting file changes is trivial - detecting changes to sections of files is hard. Keep your compact/contract extract in a separate file.*

Solution #2 — A Specification Dependency Graph

You also need to know how to rebuild your application. To do that, you need to understand what depends on what.

Front matter can work as can a database. Graph databases are the obvious choice for specification dependencies.

The simplest graph has two edges:

- `Provides` lists the routes or services a feature exposes.
- `Depends On` lists the routes or services a specification requires.

Specification Categories

Specifications can be categorized. Changes to categories earlier in the list impact downstream specifications



The Drydock specification has since been defined with additional tiers. This is the base case.

This architecture turns "what did I just invalidate" into a query. When a specification is edited, its dependent objects can be calculated. Change tickets must be tied to a parent specification and can inherit the graph from the parent. A change to a service impacts the same set of "Depends On" objects.

Solution #3: Specification Dirtying

Changes to specifications can be detected using checksums or by looking at git history. Technically, the term is to mark the file "dirty". Dirty specs need rebuild.

That is not the full definition of the problem. A changed specification does NOT necessarily force the rebuild of objects dependent on it. Only if the contract changes should the dependent specs dirty.

Changes to existing specifications should be versioned. Do not version the file yourself — have the LLM bump `Version` in the specification frontmatter when editing. A `Version` bump is a real, committed change to the file. Unlike a touched modification time, it survives a clone, it appears in a diff, and it changes the file's content hash. A version change does not invalidate the contract.

Choose a staleness signal that survives a clone. Content hashes do; git commit IDs do; timestamps do not.

Solution

Best practice: Write your change down in a ticket file with a sequence number and feed it to the LLM in a well-defined build process.

References

[1] E. Barlow. *Drydock Specification: Agile Specification-Driven Design — The SAIL Methodology for Governed Software Delivery*. Web Cloud Studio, 2026.
<https://github.com/webcloudstudio/Drydock>